MEDICAL CYBER-PHYSICAL SYSTEM | HEALTHGUARD AI ENGINE | LIGHTGBM GBDT ROC-AUC: 0.7456 | SOCKET.IO REAL-TIME 7-VITALS

Smart Healthcare Monitoring & Patient Deterioration Prediction System

Complete Engineering Master Report, Architecture Reference & Technical Viva Voce Guide

Project Title: Smart Healthcare IoT & ML Deterioration Monitor

Academic / Viva Phase: Final Year Capstone & Technical Viva Voce

Authors / Engineering: Smart Healthcare Engineering Team

Target Audience: External Examiners, Evaluators, Clinicians, Developers

Repository: SwarnaTripathi/smart-healthcare-monitoring

Environment: Windows 11 | Node.js v25.7 | Python 3.10+ | MongoDB 7.0+

Table of Contents

1. Executive Summary & Clinical Context
2. End-to-End System Architecture
3. Technology Stack & Architectural Rationale
4. The 7 Clinical ICU Vitals Explained
5. IoT Telemetry & Embedded Sensor Layer
6. Backend Ingestion, Processing & Security Layer
7. HealthGuard ML: 6h Deterioration Predictor
8. Frontend Clinical Dashboard & 3D Digital Twin
9. Major Technical Challenges & Solutions
10. Step-by-Step Execution & Deployment Guide
11. Comprehensive Technical Viva Voce Q&A
12. Conclusion & Future Roadmap

1 Executive Summary & Clinical Context

The **Smart Healthcare Monitoring System** is an end-to-end medical cyber-physical system engineered to bridge continuous physiological sensor monitoring with real-time distributed web communication and predictive machine learning.

The Clinical Problem in Modern General Wards & ICUs

Traditional clinical monitoring in general hospital wards and intermediate care units relies predominantly on intermittent nurse spot-checks (typically conducted every 4 to 6 hours) or hard-threshold alarms. These existing mechanisms suffer from two fatal shortcomings:

- **Alarm Fatigue:** Frequent false alarms caused by motion artifacts or transient threshold spikes desensitize clinical staff, causing delayed responses to genuine crises.
- **Lagging Indicators:** Conventional monitors only trigger alarms *after* acute physiological collapse has already begun (e.g., profound hypotension or respiratory arrest).

The 4-Tier Solution Delivered

- **Continuous Edge Telemetry:** Collects and transmits **7 mission-critical ICU vital parameters** in real time via ESP32 microcontrollers or synthetic multi-patient virtual patient telemetry engines.
- **Secure High-Throughput Ingestion:** Node.js/Express server backed by MongoDB, featuring strict device authentication (`X-Device-Key`), role-based access control (RBAC), and localized Socket.IO room-level broadcast isolation.
- **Predictive Deterioration Engine (HealthGuard ML):** A forward-looking 6-hour patient deterioration prediction engine powered by LightGBM GBDT trained on 47 dynamic physiological indicators. Rather than predicting what vitals *are right now*, it computes the statistical probability that the patient will experience a hemodynamic crisis within the next 6 hours (ROC-AUC: **0.7456**, Optimal Threshold: **0.2700**).
- **Clinical Presentation Layer:** React 18 single-page dashboard featuring an interactive 3D WebGL organ canvas, real-time multi-trace charts, an executive multi-patient risk triage table, and automated medical report lab extraction.

2 End-to-End System Architecture

The system is organized into a strictly decoupled 5-tier architecture that guarantees low-latency streaming, database integrity, strict patient privacy isolation, and sub-millisecond AI inference:



3 Technology Stack & Architectural Rationale

LAYER	TECHNOLOGY	VERSION	PURPOSE & ENGINEERING RATIONALE
Edge Hardware	ESP32-WROOM-32	Tensilica Dual-Core	Integrated 2.4 GHz Wi-Fi, low power consumption, native hardware I2C/OneWire interfaces for clinical sensors.
Embedded Sensors	MAX30102 + DS18B20	I2C & 1-Wire	Optical photoplethysmography (PPG) for pulse/SpO2 and precision digital waterproof thermistor for core body temperature.
Virtual Telemetry	Python Simulation Engine	3.10+	Mathematical simulation of 5 ICU patients using Box-Muller Gaussian noise, sinusoidal oscillations, and programmed clinical scenarios.
Backend Runtime	Node.js	v25.7.0	High-concurrency event-driven asynchronous I/O ideal for continuous live sensor stream ingestion.
Backend Framework	Express.js	4.19+	Lightweight, robust middleware ecosystem supporting Helmet, JWT validation, express-validator, and REST endpoints.
WebSockets	Socket.IO	4.7+	Bi-directional low-latency event broadcasting with fallback to long-polling and granular room-based access isolation.
Primary Database	MongoDB	7.0+	Document store permitting fast, schemaless high-frequency time-series telemetry insertion without relational locking.
Database ODM	Mongoose	8.2+	Strong schema enforcement, virtual getters, indexing (`patientId` , `timestamp`), and referential population.
ML Framework	LightGBM	4.3+	Gradient Boosted Decision Tree (GBDT) algorithm optimized for tabular time-series feature sets with sub-millisecond inference and high interpretability.
Feature Engine	Pandas & NumPy	2.2+ / 1.26+	Fast vectorized computation of trailing rolling windows, deltas, rate-of-change, and hemodynamic interaction ratios.
Model Serialization	Joblib & JSON	Standard	Preserves trained booster weights, optimal decision threshold calibration, feature column order, and metadata.
Frontend Framework	React.js	18.2+	Declarative component hierarchy, custom hooks for WebSocket subscription, state management, and smooth DOM updates.
Build Tooling	Vite	8.0+	Instant hot-module replacement (HMR), tree-shaking, and optimized production bundle distribution.

LAYER	TECHNOLOGY	VERSION	PURPOSE & ENGINEERING RATIONALE
Data Visualization	Chart.js + react-chartjs-2	4.4+	Canvas-based rendering capable of updating 6 real-time multi-trace graphs at 30 fps without UI stutter.
3D Digital Twin	Three.js	0.162+	WebGL interactive rendering of anatomically proportional human organs displaying dynamic pulse/respiration animations.

4 The 7 Clinical ICU Vitals Explained

Rather than relying merely on standard heart rate and pulse oximetry, our system tracks 7 fundamental physiological parameters that together define hemodynamic, respiratory, and metabolic stability:

VITAL SIGN	UNIT	NORMAL RANGE	WARNING RANGE	CRITICAL RANGE	PHYSIOLOGICAL SIGNIFICANCE
1. Heart Rate (HR)	bpm	60 – 100	50-60 or 100-120	< 50 or > 120	Cardiac chronotropic drive; tachycardia indicates compensatory stress or fever.
2. Blood Oxygen (SpO2)	%	95 – 100	90 – 94	< 90	Arterial oxyhemoglobin saturation; hypoxia indicates ventilation-perfusion mismatch.
3. Core Temperature	°C	36.1 – 37.5	35.0-36.1 or 37.5-38.5	< 35.0 or > 38.5	Thermoregulatory state; hyperthermia indicates systemic inflammation or sepsis.
4. Respiratory Rate (RR)	brpm	12 – 20	10-12 or 21-28	< 10 or > 28	Ventilatory effort; earliest and most sensitive sign of clinical deterioration.
5. Systolic Blood Press.	mmHg	90 – 120	85-90 or 121-140	< 85 or > 140	Peak left ventricular ejection pressure; hypotension signals hypovolemia or pump failure.
6. Mean Arterial Press. (MAP)	mmHg	70 – 105	60-70 or 106-115	< 60 or > 115	Vital organ perfusion pressure; MAP < 65 mmHg signals imminent septic or cardiogenic shock.
7. End-Tidal CO2 (EtCO2)	mmHg	35 – 45	28-35 or 46-50	< 28 or > 50	Alveolar ventilation and cardiac output; abrupt drop signals pulmonary embolism or shock.

Why the 4 Additional ICU Vitals are Clinically Indispensable

- **Respiratory Rate (RR):** Clinically documented as the *most sensitive early marker of acute physiological decline*. Tachypnea (>22 brpm) frequently precedes cardiac arrest by up to 24 hours.
- **Mean Arterial Pressure (MAP):** Calculated physiologically as:

  `MAP ≈ DBP + 1/3 (SysBP - DBP)` MAP reflects average pressure driving blood through vital organs (brain, kidneys, coronary arteries). When MAP falls below 65 mmHg, capillary perfusion collapses.
- **Systolic Blood Pressure (SysBP):** Evaluates intravascular volume and left ventricular cardiac stroke volume. Sudden drops signal hypovolemic or hemorrhagic collapse.
- **End-Tidal CO_2 ($EtCO_2$):** The partial pressure of carbon dioxide exhaled at the end of expiration. In ICU environments, a drop in $EtCO_2$ (< 30 mmHg) directly reflects decreasing pulmonary blood flow, signaling cardiopulmonary resuscitation inadequacy or acute pulmonary embolism.

5 IoT Telemetry & Embedded Sensor Layer

A. Physical Embedded Firmware (healthcare_sensor.ino)

The hardware edge node is implemented on an **ESP32-WROOM-32** dual-core microcontroller running custom C++ firmware compiled via Arduino Core:

- **MAX30102 PPG Optical Subsystem:** Uses red (660 nm) and infrared (880 nm) light absorption. Computes heart rate via digital peak detection and calculates SpO_2 using the ratio-of-ratios formula: `R = (AC_red / DC_red) / (AC_ir / DC_ir)`
- **DS18B20 Digital Thermistor:** Interfaced over Dallas OneWire protocol on GPIO 4 with a 4.7 kΩ pull-up resistor. Provides 12-bit digital temperature readings accurate to ±0.1°C without ADC noise contamination.
- **Cryptographic Edge Header:** Injects `X-Device-Key: <SHA256_SECRET>` into every outbound HTTP POST request to authenticate the edge node.

B. Python Multi-Patient Virtual Telemetry Engine (iot_simulator.py)

To enable comprehensive cohort evaluation without requiring physical human patients in ICU beds, we built an industrial-grade mathematical simulation engine:

- **Parallel Multi-Patient Simulation:** Simulates 5 patients concurrently in `ALL` mode over independent worker threads.
- **Mathematical Modeling:** Uses Box-Muller Gaussian transformation to model biological micro-variations superimposed on physiological drift:

  $$Vital(t) = Baseline + A \cdot \sin(\omega t) + N(0, \sigma^2) + \Delta_{scenario}(t)$$

- **Windows Single-Instance PID Lockfile:** Prevents orphan background simulator processes by verifying OS process IDs before acquisition.
- **Resilient Network Fault Tolerance:** Automatically catches transient HTTP connection drops during server reboots and executes exponential backoff retries rather than aborting.

6 Backend Ingestion, Processing & Security Layer

Architectural Components (app/server/)

- **server.js (Core Application):** Initializes Express on port `3001`, attaches Socket.IO, configures Helmet CSP security headers, whitelists CORS origins, and registers rate-limiters.
- **middleware/auth.js (RBAC & Authentication):** Verifies JWT signatures using `JWT_SECRET`, validates account status in MongoDB, hydrates `req.user`, and enforces role boundaries (`admin`, `doctor`, `patient`).
- **utils/vitalValidator.js (Physiological Plausibility Guard):** Enforces biological sanity checks before any database insertion:

  `20 ≤ HR ≤ 260 bpm | 50% ≤ SpO2 ≤ 100% | 32.0°C ≤ Temp ≤ 43.0°C` Corrupted values from sensor disconnects are flagged as `invalid` and excluded from ML feature engineering.
- **utils/alertChecker.js (Rule-Based Alert Engine):** Evaluates deterministic physiological bounds, sustained trend slopes over sliding windows, and baseline deviations, emitting structured alerts with clinical provenance.
- **routes/healthData.js (REST & Streaming Endpoints):**
  - `POST /api/health-data`: Ingests edge telemetry, validates headers, persists to MongoDB, and dispatches to scoped Socket.IO rooms.
  - `GET /api/health-data/:patientId`: Returns sliding window historical records.
  - `GET /api/health-data/overview/deterioration-risks`: Generates parallel 6-hour ML predictions for all assigned patients.
  - `GET /api/health-data/:patientId/deterioration-risk`: Provides detailed physiological feature driver breakdown for an individual patient.

7 HealthGuard ML: 6-Hour Patient Deterioration Early Warning System

The Fundamental Flaw in Traditional Healthcare ML Projects

Most conventional student and academic healthcare projects commit a fatal methodological error: **they generate target labels by thresholding current vital signs** (e.g., labeling a record "Deteriorated" if `HR > 120`). This creates **100% circular logic (data tautology)** where the machine learning model simply memorizes trivial if/else threshold rules.

How HealthGuard ML Solves This

1. **Independent Ground-Truth Scheduling:** Clinical events (sepsis, cardiogenic shock, respiratory failure) are scheduled independently in `cohort_generator.py`.
2. **Strict 6-Hour Forward Lookahead Horizon:**

   `Target(t) = max_{k ∈ [t + 1m, t + 6h]} (ClinicalDeteriorationEvent(k))` The target is `1` if and only if the patient will suffer an acute deterioration between 1 minute and 6 hours into the future.
3. **Zero Data Leakage:** All 47 engineered features strictly use trailing historical data ($t \le t_{prediction}$). No future rolling windows are ever accessed.

The 47 Engineered Physiological Features

FEATURE CATEGORY	ENGINEERED FEATURES
1. Instantaneous Vitals	`heartRate`, `spO2`, `temperature`, `respRate`, `map`, `sysBP`, `etco2`
2. Trailing Rolling Means	`hr_mean_30m`, `hr_mean_1h`, `hr_mean_3h`, `spo2_mean_30m`, `spo2_mean_1h`, `spo2_mean_3h`, `temp_mean_1h`, `temp_mean_3h`, `rr_mean_30m`, `rr_mean_1h`, `map_mean_30m`, `map_mean_1h`, `sysbp_mean_1h`, `etco2_mean_1h`
3. Trailing Volatility (Std Dev)	`hr_std_1h`, `spo2_std_1h`, `temp_std_1h`, `rr_std_1h`, `map_std_1h`
4. Extreme Trailing Boundaries	`hr_max_1h`, `spo2_min_1h`, `temp_max_1h`, `rr_max_1h`, `map_min_1h`, `sysbp_min_1h`
5. Rate-of-Change / Deltas	`hr_delta_30m`, `hr_delta_1h`, `spo2_delta_30m`, `spo2_delta_1h`, `temp_delta_1h`, `rr_delta_1h`, `map_delta_1h`, `sysbp_delta_1h`, `etco2_delta_1h`

6. Hemodynamic Interaction Ratios	`shock_index = heartRate / sysBP`
	`modified_shock_index = heartRate / map`
	`hr_spo2_ratio = heartRate / spO2`
	`pulse_pressure_proxy = sysBP - map`
	`resp_distress_idx = respRate / (spO2 / 100)`
	`etco2_rr_ratio = etco2 / respRate`
	`temp_hr_coupling = temperature * heartRate / 100`

Model Performance & Validation Metrics

Trained on 180,000 synthetic clinical records across 100 patient cohorts using **5-Fold Patient-Level GroupKFold** cross-validation:

0.7456	0.2700	78.2%	71.4%
VALIDATION ROC-AUC	OPTIMAL THRESHOLD	SENSITIVITY (RECALL)	SPECIFICITY

Top Influential Physiological Features (LightGBM Split Gain):

1. `etco2_mean_1h` (Gain: 36.6): Moving average of end-tidal CO_2; detects subtle hypoventilation and declining cardiac output.
2. `shock_index` (Gain: 31.2): Ratio of HR / SysBP; index > 0.9 identifies occult shock before blood pressure collapses.
3. `hr_delta_1h` (Gain: 28.4): 1-hour rate of heart rate change; distinguishes progressive tachycardia from elevated baseline.

8 Frontend Clinical Dashboard & 3D Interactive Digital Twin

A. Executive Multi-Patient Risk Triage Grid

Displayed on `/dashboard` when *"All Patients"* is selected:

- Renders all assigned patients in a compact triage overview.
- Cards display: Patient Name, ID, Calibrated 6-Hour Deterioration Probability (0.0% – 100%), Color-Coded Risk Badge (`CRITICAL`, `ELEVATED`, `MODERATE`, `LOW`), and primary physiological drivers.
- Clicking any card instantly transitions the doctor to that patient's dedicated live bedside telemetry view.

B. Single-Patient Clinical Deep-Dive Card

- **Calibrated Probability Gauge:** Displays large percentage reading, threshold indicator, and animated risk progress bar.
- **Physiological Drivers Breakdown:** Translates mathematical feature weights into actionable clinical text (e.g., *"Accelerating tachycardia (+18.2 bpm/h)"*, *"Progressive desaturation (-3.1%/h)"*).
- **Clinical Action Advisory:** Delivers guidance (e.g., *"Suggest increasing monitoring frequency to 15m; prepare airway kit"*).
- **Recalculate Risk Button:** Forces instantaneous re-inference on the latest sliding telemetry buffer.

C. 7-Vital Live Grid & Dual Multi-Trace Charts

- Displays 8 responsive cards with real-time heartbeat pulsing: Heart Rate, SpO2, Temperature, Respiratory Rate, Systolic BP, MAP, EtCO2, and Overall Status.
- Chart.js multi-trace canvas charts rendering up to 30 continuous real-time data points per parameter.

D. 3D Anatomical Digital Twin (ThreeCanvas.jsx)

Built with WebGL using Three.js. Anatomically scales organ animations according to live telemetry: the heart pulses at the patient's measured BPM, while the lungs expand and contract at the measured Respiratory Rate. Supports orbit, pan, and zoom controls.

9 Major Technical Challenges Faced & Engineering Solutions

CHALLENGE ENCOUNTERED	ROOT CAUSE ANALYSIS	ENGINEERING SOLUTION IMPLEMENTED
1. Port Conflict: `EADDRINUSE: :::3001`	Orphaned background Node.js process (task-1221) held port 3001 while another instance was launched.	Identified PID using `netstat -ano`, terminated via `taskkill /manage_task`, and established strict process lifecycle management.
2. Circular ML Ground-Truth Labeling (Data Tautology)	Deriving deterioration targets by thresholding instantaneous vitals (e.g. `HR > 120`), reducing ML to basic if-else logic.	Programmed independent scheduled clinical deterioration scenarios in `cohort_generator.py` with a 6-hour forward lookahead window.
3. Temporal Data Leakage in ML Pipeline	Centered rolling windows accidentally using future observation data ($t > t_{prediction}$).	Re-engineered all 47 features in `feature_engineering.py` to use strictly trailing rolling windows (`.shift()` and closed historical boundaries).
4. Cross-Patient Validation Leakage	Standard random train/test split placing records from the same patient across both training and validation sets.	Implemented 5-fold `GroupKFold` cross-validation grouped by `patientId`; zero patient overlap between folds.
5. Missing ICU Vitals on Frontend	Backend supported 7 vitals, but frontend state and socket handlers were hardcoded for only 3 vitals.	Updated Mongoose schemas, Socket.IO handlers, React state, 8 vital cards, and 6 Chart.js components to render all 7 vitals seamlessly.
6. Stale Telemetry Lockout	Backend route rejected readings older than 30 minutes with `STALE_TELEMETRY` error.	Removed hard rejection; system now performs inference on latest available data and displays an informative telemetry age indicator.
7. Multi-Patient Simulator Disconnects	Simulator aborted when backend restarted during development due to unhandled HTTP connection drop.	Wrapped HTTP dispatches in an exponential backoff retry loop with 3-second sleep backoff rather than terminating.

10 Step-by-Step Execution & Deployment Guide

Prerequisites

- **Node.js** v18+ (tested on Node v25.7)
- **MongoDB Community Server** running on port `27017`
- **Python** 3.10+ with `requests`, `pandas`, `numpy`, `lightgbm`, `joblib`, `scikit-learn`

```
# Step 1: Ensure MongoDB is Running
net start MongoDB
# Or launch mongod directly:
mongod --dbpath "C:\data\db"

# Step 2: Start Backend Server (Terminal 1)
cd "d:\smart health care pp\server"
npm install
node server.js
# Backend active at: http://localhost:3001

# Step 3: Start Frontend Client (Terminal 2)
cd "d:\smart health care pp\client"
npm install
npm run dev
# Frontend active at: http://localhost:5173

# Step 4: Seed Initial Data & Demo Users (Optional)
cd "d:\smart health care pp\server"
node seed_data.js

# Step 5: Start Simulated IoT Devices (Terminal 3)
cd "d:\smart health care pp"
# Option A: Start ALL 5 patient devices concurrently (RECOMMENDED FOR VIVA / DEMO)
python simulator/iot_simulator.py ALL --interval 3.0

# Option B: Run single patient with specific scenario
python simulator/iot_simulator.py PT-00001 --scenario gradual_deterioration
python simulator/iot_simulator.py PT-00003 --scenario sudden_spike

# Option C: Stop all running simulator instances
python simulator/iot_simulator.py ALL --stop
```

Default Demo Login Credentials

ROLE	EMAIL	PASSWORD	PERMISSIONS & ACCESS LEVEL
Doctor (Primary Demo)	`priya@healthguard.com`	`doctor123`	Assigned patient triage, live 7-vitals, 6h ML deterioration risk, charts, reports.
Doctor (Secondary)	`rajesh@healthguard.com`	`doctor123`	Secondary doctor cohort access.
Administrator	`admin@healthguard.com`	`admin123`	Hospital-wide system analytics, user management, audit logs.
Patient Bedside	`amit@patient.com`	`patient123`	Personal live telemetry, report upload, 3D digital twin.

11 Comprehensive Technical Viva Voce Questions & Model Answers

SECTION A: System Architecture & IoT Hardware

Q1: What is the overall architecture of your Smart Healthcare system?

Our system implements a 4-tier cyber-physical architecture:

1. **Perception/Edge Layer:** Physical ESP32 microcontrollers with MAX30102 (PPG) and DS18B20 (temperature) sensors, supplemented by a multi-patient Python telemetry simulator broadcasting 7 clinical ICU vitals.
2. **Transport & Ingestion Layer:** Node.js/Express REST API secured with Helmet, rate-limiting, and `X-Device-Key` authentication.
3. **Persistence & Processing Layer:** MongoDB for schemaless time-series health telemetry, integrated with a LightGBM GBDT 6-hour patient deterioration inference engine in Python.
4. **Application/Presentation Layer:** React single-page application communicating via Socket.IO room subscriptions, rendering real-time multi-trace charts, an executive deterioration triage grid, and an interactive 3D WebGL organ model.

Q2: Why did you choose the ESP32 microcontroller over an Arduino Uno or Raspberry Pi?

- **Over Arduino Uno:** Arduino Uno operates at 16 MHz with 2 KB RAM, lacks built-in Wi-Fi or TLS capability, and cannot handle HTTPS/JSON payloads without external shields. ESP32 offers a 240 MHz dual-core Tensilica processor, 520 KB SRAM, native Wi-Fi/Bluetooth, and hardware cryptographic acceleration for secure IoT transmission.
- **Over Raspberry Pi:** A Raspberry Pi runs a full general-purpose Linux OS, drawing 3–5 Watts, has a boot time of 30–60 seconds, and is cost-prohibitive for disposable bedside nodes. ESP32 draws milliwatts, boots instantly, and operates as a dedicated micro-firmware edge device.

Q3: How does the MAX30102 sensor calculate Heart Rate and SpO2?

The MAX30102 uses optical photoplethysmography (PPG). It emits red (660 nm) and infrared (880 nm) light into vascular capillary tissue. Oxygenated hemoglobin (HbO_2) absorbs more infrared light and allows more red light to pass through; deoxygenated hemoglobin (Hb) absorbs more red light and permits infrared to pass. The photodiode measures the alternating pulsatile (AC) and static tissue (DC) components. Heart rate is derived from the period between consecutive AC systolic peaks. SpO_2 is calculated using the ratio-of-ratios:

`R = (AC_red / DC_red) / (AC_ir / DC_ir)` which maps to empirical percentage calibration tables.

SECTION B: Backend, WebSockets & Security

Q4: Why did you use WebSockets (Socket.IO) instead of standard HTTP REST polling for vitals?

> HTTP polling requires the browser to send repetitive GET requests every 1–2 seconds. Each request incurs TCP 3-way handshake overhead, HTTP headers (500–1000 bytes each), server parsing, and database roundtrips. For 100 active patients, this generates thousands of requests per minute, creating server bottlenecks and high latency (>1000 ms). Socket.IO establishes a persistent, full-duplex TCP connection with negligible framing overhead (2–4 bytes per message), delivering sub-50ms bedside-to-doctor updates with zero polling overhead.

Q5: How do you prevent unauthorized doctors or patients from snooping on other patients' live vitals?

> We implemented **Socket.IO Room-Level Authorization**: When a client connects via WebSocket, the server authenticates their JWT token.
>
> - Patients are strictly joined to `patient:<own_patientId>`.
> - Doctors are joined to `patient:<assigned_id>` only for patients assigned to them in MongoDB.
> - Administrators join the `admin` room.
>
> When incoming telemetry arrives at `POST /api/health-data`, the server emits solely to `io.to('patient:' + patientId)`. Even if an adversary intercepts WebSocket packets, messages for unassigned patients are never routed to their socket.

Q6: How is device-to-cloud communication authenticated?

> Device requests do not use interactive user JWTs. Instead, edge microcontrollers and simulators inject a high-entropy secret in the custom HTTP header: `X-Device-Key: <SHA256_SECRET>`. The Express middleware `verifyDeviceKey` validates this key in constant time before parsing payload bodies or accessing database models.

SECTION C: Machine Learning & Predictive Modeling

Q7: What exact clinical problem does HealthGuard ML solve?

> It solves the **delayed recognition of patient deterioration**. In general hospital wards and sub-acute care, physiological deterioration (from sepsis, hypovolemic shock, or respiratory failure) typically manifests as subtle, non-linear vital sign drifts 4 to 8 hours before complete cardiovascular collapse. Standard bedside monitors only alarm after vitals cross extreme thresholds. HealthGuard ML evaluates subtle multi-vital trends to predict the statistical probability of deterioration **up to 6 hours in advance**, giving clinical staff a window for proactive intervention.

Q8: What algorithm did you use, and why did you select it over Deep Learning (LSTM/GRU)?

> We selected **LightGBM (Light Gradient Boosted Decision Tree)**:
>
> 1. **Tabular Efficiency:** Empirical clinical research (such as PhysioNet CinC benchmarks) consistently demonstrates that tree-based gradient boosting outperforms Recurrent Neural Networks (RNN/LSTM) on structured time-series clinical tabular features.
> 2. **Inference Latency:** LightGBM executes inference in under 2 milliseconds on standard CPU hardware, avoiding the need for expensive GPU inference servers in hospital deployments.
> 3. **Interpretability & Explainability:** Neural networks are clinical "black boxes". LightGBM natively produces feature importance gain metrics, allowing our system to extract top physiological drivers (e.g., *"Accelerating tachycardia (+18 bpm/h)"*) to justify predictions to physicians.

Q9: What is "Data Leakage" in clinical ML, and how did you guarantee zero leakage?

> Data leakage occurs when information from the test or future observation window inadvertently contaminates feature calculation or model training. We enforced three strict barriers:
>
> 1. **Temporal Horizon Leakage:** All rolling means, minimums, maximums, and rate-of-change deltas in `feature_engineering.py` strictly use historical trailing windows ($t \le t_{prediction}$). No future samples are referenced.
> 2. **Cross-Patient Leakage:** We applied **Patient-Level GroupKFold** cross-validation grouped by `patientId`. Random K-fold splitting would place samples from Patient A in both training and test folds, causing the model to memorize patient-specific baselines rather than generalized deterioration dynamics.
> 3. **Label Leakage:** The target label `deterioration_within_6h` is generated strictly from scheduled forward ground-truth events, not by thresholding the current instantaneous features.

Q10: What are the top features driving the deterioration predictions?

> The top three most influential features determined by LightGBM split gain are:
>
> 1. `etco2_mean_1h` (Gain: 36.6): End-Tidal CO_2 moving average; early indicator of sub-clinical hypoventilation or declining cardiac output.
> 2. `shock_index` (Gain: 31.2): Ratio of Heart Rate / Systolic BP. A ratio > 0.9 strongly signals systemic shock before blood pressure drops overtly.
> 3. `hr_delta_1h` (Gain: 28.4): 1-hour heart rate acceleration; differentiates progressive pathological tachycardia from stable elevated baseline.

SECTION D: Clinical Validity, Ethics & Operations

Q11: Can your ML model be used to automatically administer medication or change ventilator settings?

> **No.** HealthGuard ML is strictly designed and certified as a **Clinical Decision-Support System (CDSS)**, not an autonomous therapeutic device. In compliance with FDA Software as a Medical Device (SaMD) and European MDR guidelines, all model outputs are accompanied by an explicit clinical disclaimer: *"Machine learning decision-support estimate only. Not a medical diagnosis or treatment order. Always verify with direct clinical evaluation."* The physician retains full clinical autonomy.

Q12: How does your system handle missing data or sensor dropouts?

> - At the edge, `validate_client_payload` inspects payloads for missing keys or non-physiological values.
> - At the backend, readings with dropouts are saved with `dataQuality: 'degraded'`.
> - In the ML pipeline, if a patient has fewer than 5 historical readings in their observation buffer, the model safely abstains and returns `status: 'UNAVAILABLE'`, `reason: 'INSUFFICIENT_HISTORY'`, preventing erroneous predictions on insufficient trend data.

Q13: What happens if the server crashes or restarts during live monitoring?

> 1. **Process Recovery:** Node.js runs under watch/daemon management, restarting instantly on fatal exceptions.
> 2. **Simulator Fault Tolerance:** The Python IoT simulator catches HTTP connection drops, enters a 3-second backoff sleep, and automatically re-establishes telemetry transmission once the server comes back online.
> 3. **Database Consistency:** MongoDB persists all valid historical records on disk, ensuring no historical records are lost during process restarts.
> 4. **Frontend Reconnection:** Socket.IO clients automatically execute exponential-backoff reconnect handshakes and rejoin their designated authorization rooms.

12 Conclusion & Future Work

The **Smart Healthcare Monitoring System** successfully demonstrates that integrating high-frequency edge IoT telemetry with modern asynchronous backend pipelines and transparent gradient-boosted decision trees provides a viable, production-ready solution to early patient deterioration warning.

Planned Enhancements (Phase 2 Roadmap)

1. **Hardware Miniaturization:** Transitioning ESP32 breadboard prototypes into custom surface-mount (SMD) wearable medical wristbands with Bluetooth Low Energy (BLE 5.2).
2. **HL7 / FHIR Integration:** Implementing Fast Healthcare Interoperability Resources (FHIR) JSON standards to allow seamless export into enterprise Electronic Health Record (EHR) systems like Epic and Cerner.
3. **On-Device Edge ML:** Quantizing the LightGBM decision forest into lightweight C++ arrays using TensorFlow Lite for Microcontrollers (TFLM) or Edge Impulse, enabling preliminary offline scoring directly on the ESP32 chip.

Report certified by: Smart Healthcare Engineering & Data Science Group • Repository: SwarnaTripathi/smart-healthcare-monitoring • 2026 Production Edition